

Introduction to Big Data

Chapter 3

Classification and regression trees (CART)

Anne RUIZ-GAZEN, TSE-UT1C

Note that these slides are based on several lectures notes and slides available on the internet.

January 25, 2021

1 Introduction

- What is a binary tree?
- Pros and cons

2 Procedure

- Basics
- Notations
- Building a split
- Pruning the tree

1 Introduction

- What is a binary tree?
- Pros and cons

2 Procedure

- Basics
- Notations
- Building a split
- Pruning the tree

Introduction

- Also called **Decision** trees. One variable to explain (supervised method contrarily to PCA and Clustering that are unsupervised methods, part of machine learning: algorithms that “learn” from the data):
 - ▶ qualitative (categorical) → **classification**
 - ▶ or quantitative (numerical) → **regression**.

Predictors are either numerical or categorical.

- A decision tree is defined using a **training** sample and tested on a **test** sample (split the original sample in two samples).
- The method is nonlinear.
- Different algorithms exist. We consider only **CART** developed by Breiman, Friedman, Olsen and Stone (1984): “Classification And Regression Trees”.

1 Introduction

- What is a binary tree?
- Pros and coins

2 Procedure

- Basics
- Notations
- Building a split
- Pruning the tree

Examples

- 1 Predict the **land use** of a given area (agriculture, forest,...) given satellite information, meteorological data, socio-economic information, prices information,....
- 2 Predict high risk for **heart attack**:
 - ▶ University of California: a study into patients after admission for a heart attack.
 - ▶ 19 variables collected during the first 24 hours for 215 patients (for those who survived the 24 hours)
 - ▶ Question: Can the high risk (will not survive 30 days) patients be identified?
- 3 Determine how **computer performance** is related to a number of variables which describe the features of a PC (the size of the cache, the cycle time of the computer, the memory size and the number of channels. Both the last two were not measured but minimum and maximum values obtained).

Examples

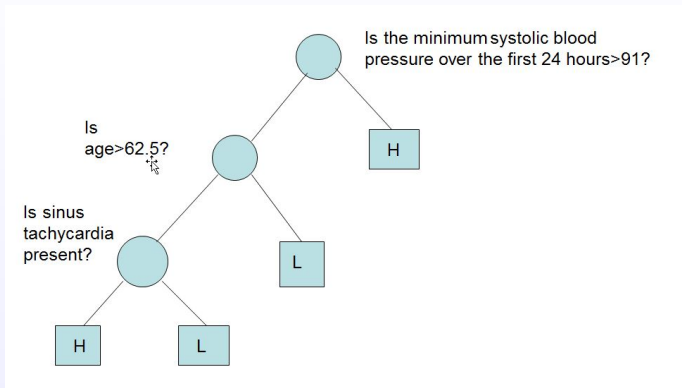


Figure: A binary tree

Examples

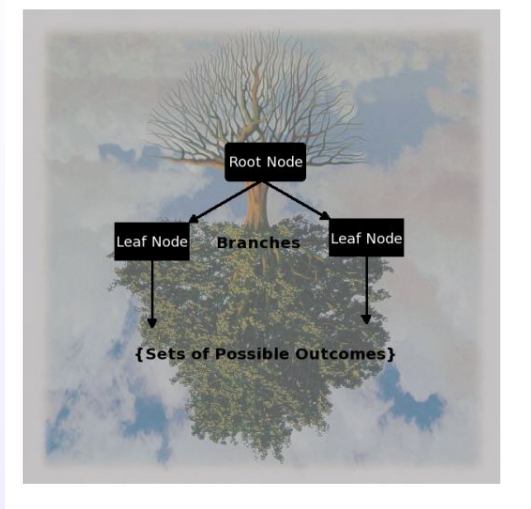


Figure: A true tree

What is a binary tree?

- The top, or first node, is called the **root node**.
- The last level of nodes are the **leaf nodes** and contain the final classification.
- The intermediate nodes are **child nodes**.
- **Binary trees** (two child nodes) are the most popular type of tree. However, M-ary trees (M branches at each node) are possible.
- Nodes contain one question. In a binary tree, by convention if the answer to a question is “yes”, the left branch is selected. Note that the same question can appear in multiple places in the tree.
- Key questions include **how to grow the tree, how to stop growing, and how to prune the tree** to increase generalization (good results should not be obtained only on the training set).

1 Introduction

- What is a binary tree?
- Pros and coins

2 Procedure

- Basics
- Notations
- Building a split
- Pruning the tree

When using classification or regression trees?

Decision trees are well designed for

- problems with a **big number of variables** (a variable selection is included in the method);
- providing an **intuitive interpretation** with a simple graphic;
- solving **either regression and classification problems with quantitative or qualitative predictors**, hence the name **CART**.

but they face several difficulties:

- they **need a large training dataset** to be efficient;
- as they select the variables hierarchically, they can **fail to find a global optimum**.

1 Introduction

- What is a binary tree?
- Pros and cons

2 Procedure

- Basics
- Notations
- Building a split
- Pruning the tree

Basics about classification and regression trees

The algorithm is based on two steps:

- a **recursive binary partitioning** of the data space spanned by the predictor variables into increasingly homogeneous groups with respect to the response variable as measured by:
 - ▶ For regression: Mean squared error.
 - ▶ For classification (with classes $k = 1, 2, \dots$): the Gini index (or entropy or misclassification rate).
- a **pruning** algorithm of the tree when it is too complex.

2

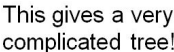


Figure: A tree too complex $\rightarrow$ pruning

Basics about classification and regression trees

The subgroups of data created by the partitioning are called **nodes**.

- For **regression** trees the predicted (fitted) value at a node is just the **average** value of the response variable for all observations in that node.
- For **classification** trees the predicted value is the class in which there is the **largest** number of the observations at the node.
- For **classification** trees one also gets the estimated probability of membership in each of the classes from the proportion of the observations at the node in each of the classes.

Basics about classification and regression trees

The subgroups of data created by the partitioning are called **nodes**.

- At each step, an optimization is carried out to select:
 - ▶ A node,
 - ▶ A predictor variable,
 - ▶ A cut-off value if the predictor variable is numerical, or a group of modalities if the predictor variable is categorical,
- That maximizes the (percentage) decrease in the objective function (homogeneity measure).

1 Introduction

- What is a binary tree?
- Pros and cons

2 Procedure

- Basics
- **Notations**
- Building a split
- Pruning the tree

Notations

The data is built from a pair of random variables, (X, Y) , where X is made of p qualitative or quantitative predictors, $X^1, \dots, X^p$, and Y is a qualitative (classification) or quantitative (regression) variable to predict from X .

n i.i.d. observations of (X, Y) , $(x_1, y_1), \dots, (x_n, y_n)$, are given.

As explained previously, building a decision tree aims at finding a series of:

- **nodes**, which are defined by the choice of a variable, X^k ,
- **splits** deduced from this variable and a threshold, τ ,

dividing the training sample into two subsamples: $\{i : x_i^k < \tau\}$ and $\{i : x_i^k > \tau\}$ (or in two groups of modalities of x is categorical).

Notations

In the following, we will note the nodes with the following convention:

- the **root** will be denoted by $\mathcal{N}^0$;
- the **child nodes of the root** will be denoted by $\mathcal{N}_k^1$ with $k = 1, 2$;
- the **child nodes of $\mathcal{N}_k^1$** will be denoted by $\mathcal{N}_{2k-1+j}^2$ with $k = 1, 2$ and $j = 0, 1$ (hence these nodes are indexed from 1 to 4);
- ...
- the **nodes at step m** will be denoted by $\mathcal{N}_k^m$ for $k = 1, \dots, 2^m$ (where, eventually, some of the indexes can be unused): $\mathcal{N}_1^m$ and $\mathcal{N}_2^m$ are the child nodes of $\mathcal{N}_1^{m-1}$, $\mathcal{N}_3^m$ and $\mathcal{N}_4^m$ are the child nodes of $\mathcal{N}_2^{m-1}$, ...
- ...

The need of an homogeneity criterion

When splitting $(y_i)_i$ into two groups, we aim at having two non empty groups with Y values as **homogeneous** as possible.

Then, we have to define an homogeneity criterion for each node, i.e. a function

$$H : \mathcal{N} \rightarrow \mathbb{R}_+$$

that is

- **null if and only if** all the individuals that belong to the node have the same Y value;
- **large when** all the Y values are very different.

The split of a node $\mathcal{N}_k^m$ is chosen, among all possible splits, by **minimizing** the sum of the homogeneity of the corresponding child nodes:

$$\arg \max_{\text{splits of } \mathcal{N}_k^m} H(\mathcal{N}_k^m) - (n_1 H(\mathcal{N}_{2k-1}^{m+1}) + n_2 H(\mathcal{N}_{2k}^{m+1})) / (n_1 + n_2).$$

Homogeneity of a split

Consider a given node, $\mathcal{N}_k^m$, that has to be split into two classes (two new child nodes) and denote by

- $n_1 = \sum_{i=1}^n \mathbb{I}_{\{y_i \in \mathcal{N}_{2k-1}^{m+1}\}}$ and $n_2 = \sum_{i=1}^n \mathbb{I}_{\{y_i \in \mathcal{N}_{2k}^{m+1}\}}$;
- for $j \in \{-1, 0\}$ and a regression tree
 $H(\mathcal{N}_{2k+j}^{m+1}) = \sum_{i=1}^n (y_i - \mathcal{Y}_{2k+j})^2 \mathbb{I}_{\{y_i \in \mathcal{N}_{2k+j}^{m+1}\}}$ where $\mathcal{Y}_{2k+j}$ is the predicted value in node $\mathcal{N}_{2k+j}^{m+1}$: $\mathcal{Y}_{2k+j} = \sum_{i=1}^n y_i \mathbb{I}_{\{y_i \in \mathcal{N}_{2k+j}^{m+1}\}}$. The sum of squared criterion is replaced by the Gini index for classification trees

The growing of the tree is stopped if the **obtained node is homogeneous** or if the **number of observations in the nodes is smaller than a fixed number** (generally chosen between 1 and 5).

Building a split: small classification example detailed

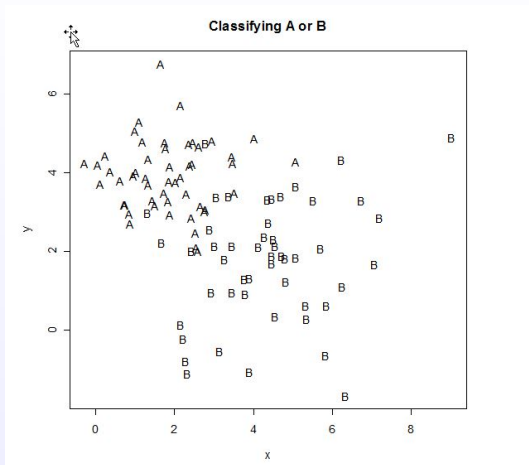


Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

- In order to split a node, we need to choose a criterion of inhomogeneity or measure of **impurity**.
- The measure should be at a maximum when a node is equally divided amongst the two classes.
- The impurity should be zero if the node is all one class.
- The Gini index is the most widely used measure of impurity (at least by CART):

$$1 - p_A^2 - p_B^2$$

where p_1 and p_2 are respectively the proportion of observations in class A and in class B for a given node

- we compute the decrease of the Gini index when splitting a given node in two child nodes by calculating the Gini index at the given node and subtract one half of the sum of the Gini indices at the child nodes as illustrated in the small example below.

Building a split: small classification example detailed

- We select the split (a variable and a threshold) that most decreases the Gini Index. This is done over all possible places for a split and all possible variables to split.
- There are two possible variables to split on and each of those can split for a range of values of c i.e.: $x < c$ or $y < c$.
- We keep splitting until the terminal nodes have very few cases or are all pure. Note that this seems an unsatisfactory answer to when to stop growing a tree, but it was realized that the best approach is to grow a larger tree than required and then to prune it.

Building a split: small classification example detailed

			Split= 2.81			
x	y	Class	A	B	A	B
2.61	2.02	A	1	0	0	0
2.57	2.10	A	1	0	0	0
2.85	2.46	B	0	0	0	1
2.45	2.85	A	1	0	0	0
2.76	3.00	A	1	0	0	0
2.82	3.07	A	0	0	1	0
2.68	3.13	B	0	1	0	0

Etc.

Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

Top	50	50	100	0.5	0.5	0.25	0.25	0.5	
Split Left	44	7	51	0.86	0.14	0.74	0.02	0.24	0.12
Split Right	6	43	49	0.12	0.88	0.01	0.77	0.21	0.11
								Sum=	0.23
								Change in	0.27
								Gini Index	

Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

Then use Data table to find the best value for a split.

Split	Change
	0.27
1.5	0.10
1.6	0.11
1.7	0.12
1.8	0.13
1.9	0.16
2	0.16
2.1	0.17
2.2	0.17
2.3	0.15

Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

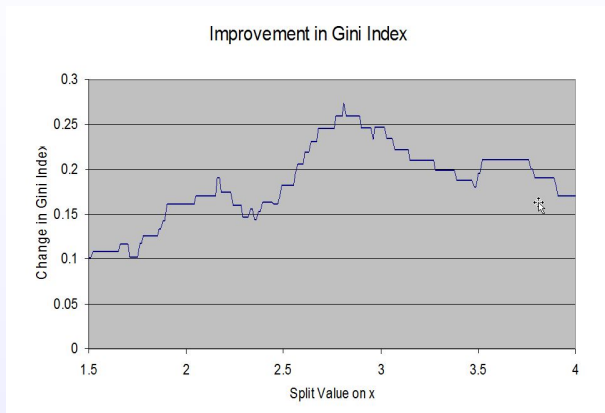


Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

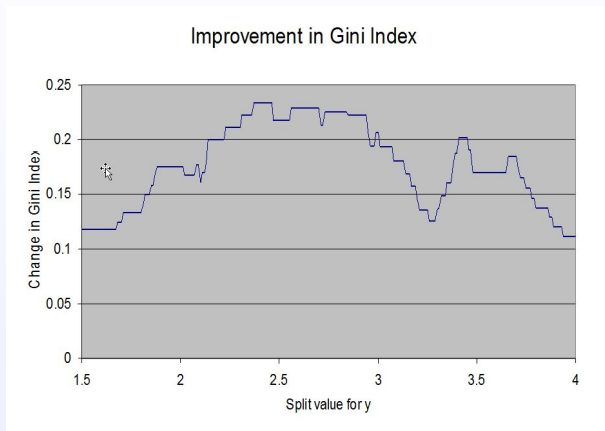


Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

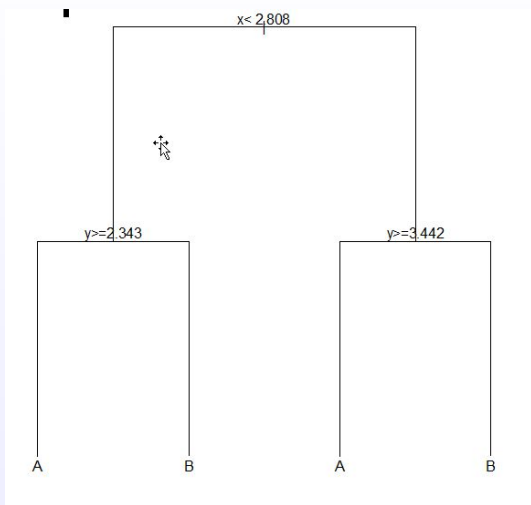


Figure: A two-classes variable to predict with two quantitative predictors x and y

Building a split: small classification example detailed

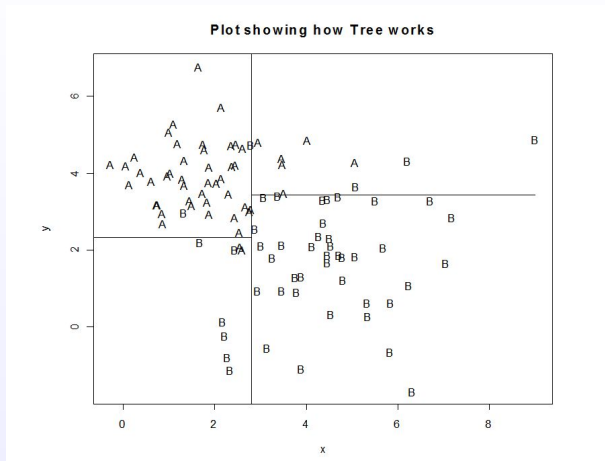


Figure: A two-classes variable to predict with two quantitative predictors x and y

1 Introduction

- What is a binary tree?
- Pros and cons

2 Procedure

- Basics
- Notations
- Building a split
- Pruning the tree

Why and how pruning?

As said earlier it has been found that the best method of arriving at a suitable size for the tree is to grow an overly complex one then to **prune** it back. The pruning is based on the **misclassification rate** (number of observations misclassified divided by the total number of observations, see also **confusion table**). However the error rate will always drop (or at least not increase) with every split. This does not mean however that the error rate on the test data will improve.

Why and how pruning?

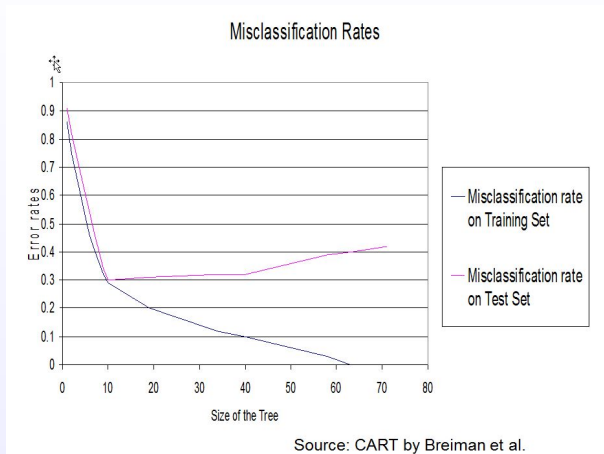


Figure: Why pruning?

Why and how pruning?

To overcome the **overfitting** problem (good results on the training set but bad results on the test set).

A solution is to choose one of the subtree of the maximal tree obtained by **iterative pruning** and to choose the **optimal subtree** for a given quality criterion. This step-by-step methodology do not necessarily lead to a global optimal subtree but it is a computationally plausible solution.

A penalized criterion

The idea of pruning is to estimate an **error criterion penalized by the complexity of the model**.

$$L\mathcal{T} = \sum_{k=1}^K L\mathcal{F}_k \quad \text{where } L\mathcal{F}_k = \sum_{i: x_i \in \mathcal{F}_k} (y_i - \mathcal{Y}_k)^2$$

for the regression case and the misclassification rate for the classification case.

Hence, a penalized version of the error that takes into account the complexity of the tree can be defined through:

$$L_\gamma \mathcal{T} = L\mathcal{T} + \gamma K.$$

where K is the size of the tree (number of nodes).

Iterative pruning

When $\gamma = 0$, $L_\gamma \mathcal{T} = L\mathcal{T}$ and hence the tree optimizing this criterion is $\mathcal{T}$ (which has been designed for).

When γ is increasing, one of the nodes' split, $\mathcal{S}_j$ appears such that:

$$L_\gamma \mathcal{T} > L_\gamma \mathcal{T}^{-\mathcal{S}_j}$$

where $\mathcal{T}^{-\mathcal{S}_j}$ is the tree where the split $\mathcal{S}_j$ has been removed. Let us call $\mathcal{T}_{K-1}$ this new tree.

This process is iterated to obtain a **sequence of trees**

$$\mathcal{T} \supset \mathcal{T}_{K-1} \supset \dots \mathcal{T}_1$$

where $\mathcal{T}_1$ is the tree restricted to its root.

Choosing the optimal subtree

The optimal subtree is chosen by validation or cross-validation by the following algorithm:

Algorithm for cross validation choice

- 1 Build the maximal tree, $\mathcal{T}$
- 2 Build the sequence of subtrees $(\mathcal{T}_k)_{k=1,\dots,K}$
- 3 By validation (or cross validation), evaluate the errors $L\mathcal{T}_k$ for $k = 1, \dots, K$
- 4 Build the graphic of $L\mathcal{T}_k$ in function of $k = 1, \dots, K$
- 5 Finally, choose k which minimizes $L\mathcal{T}_k$ (cp= complexity parameter)

Pruning a tree: small classification example detailed

- One version of the method carries out a 10 fold cross validation where the data is divided into 10 subsets of equal size (at random) and then the tree is grown leaving out one of the subsets and the performance assessed on the subset left out from growing the tree. This is done for each of the 10 sets. The average performance is then assessed.
- This is all done by the command “rpart” in R and the results can be accessed using “printcp” and “plotcp”. We can then use this information to decide how complex (determined by the size of cp) the tree needs to be. The possible rules are to minimize the cross validation relative error (xerror), or to use the “1-SE rule” which uses the largest value of cp with the “xerror” within one standard deviation of the minimum. This is preferred by Breiman et al. (see the dashed line in the “plotcp” function).

Pruning a tree: small classification example detailed

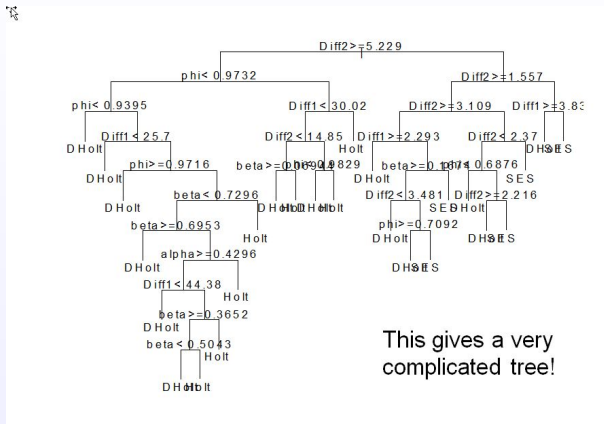


Figure: Small classification example

The data are not detailed.

Pruning a tree: small classification example detailed

Classification tree:

```
rpart(formula = Model ~ Diff1 + Diff2 + alpha + beta + phi, data = expsmooth,  
      cp = 0.001)
```

Variables actually used in tree construction:

```
[1] alpha beta Diff1 Diff2 phi
```

Root node error: 2000/3000 = 0.66667

n= 3000

	CP	nsplit	rel error	xerror	xstd
1	0.4790000	0	1.0000	1.0365	0.012655
2	0.2090000	1	0.5210	0.5245	0.013059
3	0.0080000	2	0.3120	0.3250	0.011282
4	0.0040000	4	0.2960	0.3050	0.011022
5	0.0035000	5	0.2920	0.3115	0.011109
6	0.0025000	8	0.2810	0.3120	0.011115
7	0.0022500	9	0.2785	0.3085	0.011069
8	0.0020000	13	0.2675	0.3105	0.011096
9	0.0017500	16	0.2615	0.3075	0.011056
10	0.0016667	20	0.2545	0.3105	0.011096
11	0.0012500	23	0.2495	0.3175	0.011187
12	0.0010000	25	0.2470	0.3195	0.011213

Figure: Small classification example

Pruning a tree: small classification example detailed

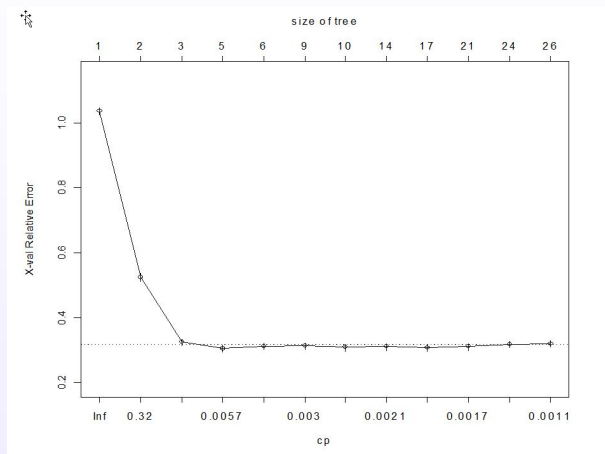


Figure: Small classification example

Pruning a tree: small regression example detailed

Determine how **computer performance** is related to a number of variables which describe the features of a PC (the size of the cache, the cycle time of the computer, the memory size and the number of channels. Both the last two were not measured but minimum and maximum values obtained).

Pruning a tree: small regression example detailed

This gave the following tree:

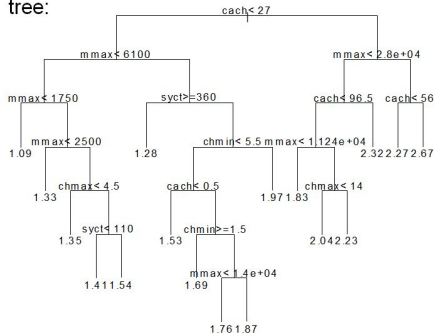


Figure: Small regression example

Pruning a tree: small regression example detailed

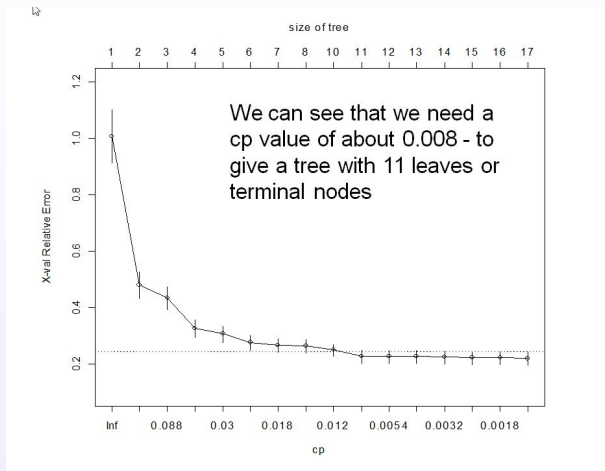


Figure: Small regression example

Pruning a tree: small regression example detailed

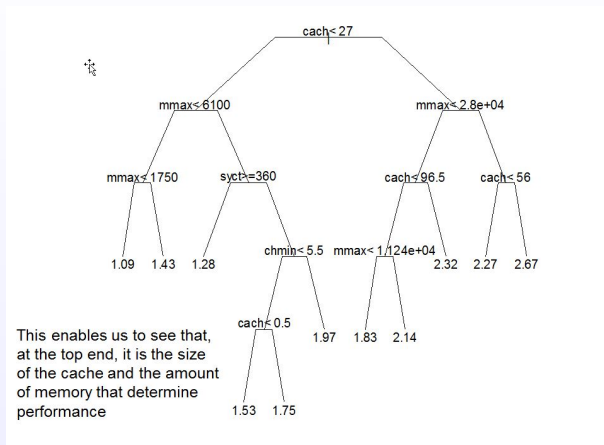


Figure: Small regression example